高速智能仓储二维码识别系统

摘要

随着智能仓储与物流向自动化、高速化发展，货物出入库及分拣对二维码读取的实时性、准确性与稳定性提出更高要求。传统工业相机结合 CPU 工控机的视觉方案在连续高速处理时面临计算负载、设备成本与功耗等挑战。针对上述问题，本项目基于 **Zynq-7020/AX7021B** 设计高速智能仓储二维码识别系统，采用 **PL-PS 协同架构**，在单板完成摄像头采集、结构候选提取、内容解码与 HDMI 视频输出，面向货物出入库和输送分拣提供边缘信息采集能力。

本项目以开源 quirc 二维码识别算法为基础进行软硬件协同适配，将摄像头图像分为**识别与显示两条并行路径**。识别采用**结构特征引导的候选检测方法**，结合 Otsu 阈值分割与四方向游程检测，依据二维码定位图案的黑白比例及几何关系筛选候选区域 ROI，缩小 PS 的精确定位范围。针对透视变形，采用逐系数正负试探与评分比较优化采样网格；PS 预生成双精度候选系数树，PL 按迭代依赖顺序执行**整网格评分及接受决策**，减少软硬件交互开销，再由 PS 依据优化后的网格完成模块采样、纠错和内容恢复。显示采用**独立彩色通路与 OSD 叠加**，由 PS 根据解码内容及目标位置生成框线和文字标签，PL 将完整标注同步叠加至 HDMI 画面，使识别与视频显示并行运行。

虚拟网页仿真测试覆盖 5000 个二维码，内容召回率为 99.94%，加权识别处理帧率为 27.939 FPS；首轮配对测试中，处理帧率约为纯 PS 方案的 9.13 倍。真实传送带测试的 150 个二维码全部正确读取。结果表明，PL-PS 协同能够提升识别吞吐和运动目标的内容读取完整性。

关键词：Zynq；二维码高速识别；PL 与 PS 异构加速；Linux 驱动；透视网格加速

目录

摘要 1
第一部分　作品概述 3
1.1 选题背景 3
1.2 功能与特性 3
1.3 应用领域 3
1.4 主要技术特点与创新 4
1.5 主要性能指标 5
1.6 设计流程 5
第二部分　系统组成及功能说明 5
2.1 整体介绍 5
2.2 硬件系统介绍 6
2.2.1 硬件整体介绍 6
2.2.2 PL 各模块介绍 7
2.3 软件系统介绍 10
2.3.1 软件整体介绍 10
2.3.2 软件各模块介绍 10
2.4 大模型协作与技能包提炼 15
2.4.1 协作方式与记录 15
2.4.2 代表性提示与纠错案例 15
2.4.3 技能的提炼与复用 17
第三部分　完成情况及性能参数 18
3.1 整体介绍 18
3.2 工程成果 18
3.2.1 软件成果 18
3.3 优化过程与资源对比 19
3.3.1 图像采集等待的定位与优化 19
3.3.2 从单次评分加速到整网格协同 19
3.3.3 独立显示优化 20
3.3.4 分项性能与资源成本 20
3.3.5 同类平台识别速度对比 22
3.4 性能测试与加速效果验证 23
3.4.1 测试场景与统计方法 23
3.4.2 累计结果与速度分项 24
3.4.3 首轮 1000 次的纯 PS 配对对照 26
3.4.4 真实传送带测试 27
第四部分　总结 28
4.1 可扩展之处 28
4.2 不足之处 28
4.3 心得体会 28
第五部分　参考文献 29
附录 A 复现说明 30
A.1 硬件与工程材料 30
A.2 SD 系统备份与空白卡恢复 31
A.3 环境准备与源码校验 31
A.4 HLS 与 Vivado 硬件重建 32
A.5 PS 应用和 Linux 驱动构建 33
A.6 安装启动与运行核验 33
A.7 功能验证与虚拟网页仿真测试 34
A.8 统计比对与资料保存 34
附录 B 完整硬件连接图 35

第一部分　作品概述

1.1 选题背景

智能仓储中的货物出入库、输送分拣，依赖对运动目标信息的连续读取。二维码成本低、易部署且便于承载编码信息，已成为仓储追溯的重要载体。但二维码在相机视场停留时间有限，系统既要完成对图像采集、灰度化和二值化，又要对二维码精确定位和识别。多目标并行、平面旋转和透视变化会增加定位及网格恢复负载，采集、识别和显示为处理器与内存带宽带来压力，现有识别设备在动态多码场景下仍面临实时性、吞吐和能效压力。

已有研究从算法和计算平台两方面展开探索。Lam 与 Sum 提出面向 FPGA 实现的二维码检测方法[2]；结合深度学习检测、去模糊与 Zynq 平台的方案增强了复杂图像处理能力[4]。开源 quirc 提供 C 语言定位、模块采样和纠错实现，本项目在其源码基础上进行适配，便于资源受限平台嵌入式集成[6]。陈荣军等采用改进 CORDIC 实现 FPGA 二维码定位与旋转校正[1]，Lam 与 Sum 提出面向 FPGA 的快速检测方法[2]；方园园和 Gu 等分别基于 Zynq 开展运动模糊恢复，以及深度学习检测与去模糊识别[3–4]。但局部模块性能和单图处理指标，尚不足以评价连续多码输入下的完整识别吞吐。本项目进一步加速整网格评分与接受决策，并结合采集识别并行、源帧关联和独立显示，减少计算与等待开销，保持图像、结果与标注对应。

1.2 功能与特性

本项目实现连续采集、多二维码识别、结果关联和 HDMI 视频输出叠加显示。摄像头输入分辨率为 1280×720，PL 最多输出 30 个候选区域，PS 按优先级执行局部识别，单次成功结果及显示标注最多 8 个。输出包含完整内容、原图位置、绿色外接框和短标签，便于将读取结果与画面中的具体目标对应。

连续帧中的同一目标通过内容与位置关联，避免重复输出；不同位置的相同内容保留独立身份。Linux 提供统一调度、状态查询和日志入口，使现场操作能够同时观察识别状态与视频反馈。

1.3 应用领域

本项目面向货物出入库和输送分拣，能够在板端完成摄像头采集、二维码识

别及可视化反馈。对于空间和算力受限的边缘节点，该集成方式可减少对独立高能耗工控机的依赖，具有降低部署成本与运行功耗的潜力。

系统分别通过虚拟网页仿真测试和真实传送带测试验证识别能力。前者由摄像头拍摄屏幕中的运动二维码，通过重复设置内容、尺寸、角度及速度建立测试真值；后者使用实验室传送带输送实际二维码，测试平台见图 1-1。



图 1-1 系统实物组成与真实传送带测试平台

1.4 主要技术特点与创新

（1）**采集与识别并行**。Linux 驱动提前下一空闲帧缓冲区配置采集 DMA，使硬件采集准备与软件处理周期重叠；以私有灰度快照储存软件缓存，与摄像头输入图像分离，并由 PL 独立输出彩色预览，减少 ARM 显示负担，提高各处理环节的并行利用程度。

（2）**HLS 透视网格优化加速器**。将透视优化中的整网格评分与接受决策集成为单个硬件作业 IP，利用样本流水和输入复用降低重复计算及控制开销；通过双精度候选系数保持原有数值语义，兼顾迭代依赖、正确性与加速效果。

（3）**源帧关联与标注更新**。贯通图像、候选区域、评分作业和成功结果的来

源身份，以缓冲所有权约束数据复用；框线与文字按完整场景同步提交，使多目标内容、空间位置及显示标注保持对应。

1.5 主要性能指标

系统支持 1280×720 分辨率图像输入。在 K4 至 K6[1]三档目标尺度和五档运动速度下开展虚拟网页仿真测试，内容召回率与识别处理帧率见图 3-4、图 3-5，与纯 PS 方案的配对比较见图 3-6、图 3-7。独立彩色预览的新图像更新率约为 30.001 FPS，HDMI 输出刷新率为 60 Hz。

1.6 设计流程

设计首先围绕运动多码场景明确连续采集、内容识别和视频标注需求，建立可运行基线，对采集等待、透视评分及显示开销进行分段测量，据此确定软硬件分工：PL 承担像素流处理、重复评分和彩色预览，PS 保留精确定位、最终采样、纠错及结果管理。

实现阶段采用 RTL 构建视频与候选通路，采用 HLS 实现整网格评分，并通过 Linux 驱动和应用完成任务调度与数据交接。软硬件调试时，核对图像、候选区域与处理结果的来源，规范缓冲区的读写交接和任务完成确认流程，并通过功能比对、仿真及板端测试验证数据对应关系与处理流程的正确性。最终通过虚拟网页仿真测试统计内容召回率与处理帧率，并设置纯 PS 配对基线评估协同加速效果；另通过真实传送带测试验证实际输送场景中的内容读取能力，两类测试分别统计。

第二部分　系统组成及功能说明

2.1 整体介绍

系统由 OV5640 相机、AX7021B 开发板、HDMI 显示器、SD 卡和调试 PC 主机组成。识别与显示均在板端完成；主机通过以太网建立 SSH 连接，或通过 UART 串口登录板端，进行运行控制、查看识别结果与日志，并生成虚拟网页仿真测试场景。整体结构包含独立 HDMI 彩色预览、灰度识别和透视网格优化三

[1]K 表示源画布上二维码单个模块（黑白网格单元）的边长，单位为像素；K4、K5、K6 分别对应 4、5、6 像素/模块。

条数据路径，如图 2-1 所示。

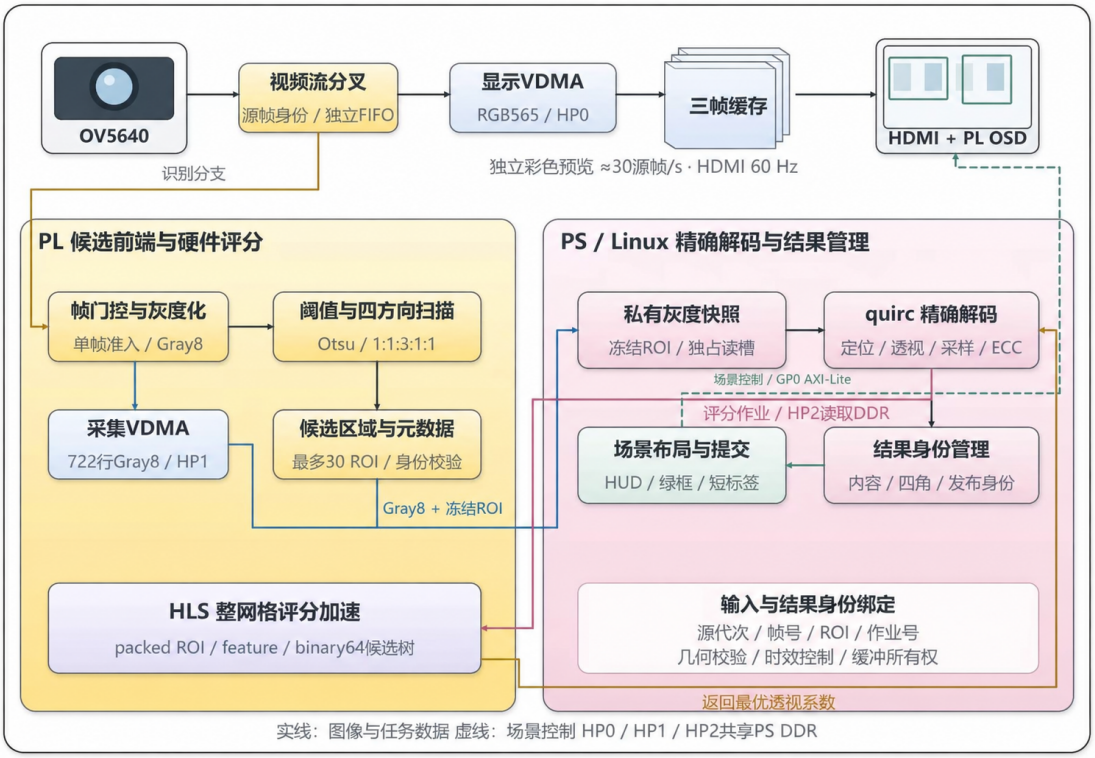


图 2-1 系统总体结构及数据流设计示意

2.2 硬件系统介绍

2.2.1 硬件整体介绍

AX7021B 搭载 XC7Z020CLG484-2，PS 包含双 Cortex-A9 并运行 Linux，PL 集成视频采集前端、基于 HLS 的透视网格校正加速器以及 HDMI 显示输出通路。OV5640 摄像头模组在本系统中仅使用一路图像。板载 DDR 同时存放帧图像和评分输入，主机通过以太网使用 SSH，或通过 UART 串口访问板端。硬件功能框图见图 2-2，主要部件见表 2-1。

表 2-1 硬件组成与接口

部件	作用	关键接口或数据
OV5640 相机模组	单路图像采集	8 位 DVP、PCLK、HREF、VSYNC、SCCB
Zynq PS	Linux 与识别控制	Cortex-A9、DDR、GP0、HP 端口、IRQ
Zynq PL	前端处理、评分及显示	AXI4-Stream、AXI-Lite、BRAM、DSP
HDMI 发送链路	彩色画面输出	RGB24、DE、HS、VS、像素时钟、I^2C
SD 卡	启动与根文件系统	BOOT.BIN、image.ub、boot.scr、ext4

部件	作用	关键接口或数据
调试主机	运行控制、结果与日志查看、测试场景生成	以太网/SSH；UART 串口（115200、8N1）；浏览器场景

2.2.2 PL 各模块介绍

PL 端构建连续像素处理、整网格优化与独立彩色显示协同运行的硬件架构。摄像头图像经独立流控分为识别与预览两路，识别前端流水完成灰度化、Otsu 阈值计算及水平、垂直和双对角扫描，再通过定位图案聚类、三点几何筛选与区域合并生成候选区域，缩小软件搜索范围。针对透视校正热点，将 81 次关联评分及接受决策组织为单个硬件作业，复用片上二值图像，采用双精度样本流水，并沿 PS 预生成的候选系数树保持迭代顺序与数值语义。显示侧通过彩色帧缓冲独立输出 HDMI，无需等待解码完成；框线与文字在垂直消隐期整组切换。该架构通过像素流水、整网格作业与独立预览，减少重复访存、控制往返及环节等待，支持采集、识别与显示并行推进。

（1）**DVP 接收**。通过 SCCB 配置 OV5640 的分辨率与 RGB565 输出模式，接收逻辑依据像素时钟、行有效和帧同步恢复图像。每两个有效字节组成一个 16 位像素，并附加行帧边界信息，供后级按照完整图像组织处理。

（2）**分流与准入**。彩色预览和灰度识别分支分别配置 FIFO 与流控，使识别等待不直接阻塞预览。识别仅在空闲缓冲准备完成后从帧首接收完整图像；若发生异常，则在帧边界重新建立同步，避免将两幅图像的局部像素拼成一个有效帧。

（3）**灰度化与阈值**。RGB565 分量扩展至 8 位后，转换为 8 位灰度图像 Gray8［式（2-1）］。77、150、29 为亮度权重的整数近似，除以 256 可通过移位实现，适于像素流水处理。Otsu 根据中央采样窗口的灰度直方图，按式（2-2）寻找类间方差最大的阈值[5]；帧首锁存最近有效结果，避免同一帧扫描中途改变二值判断。

$$Y = \left\lfloor \frac{77R + 150G + 29B}{256} \right\rfloor \tag{2-1}$$

$$\sigma^2(t) = \omega_0(t)\omega_1(t)[\mu_0(t) - \mu_1(t)]^2,$$
$$T^* = \arg\max_{0 \le t \le 254} \sigma^2(t) \tag{2-2}$$

式中 R、G、B 为 8 位颜色分量，ω_0、ω_1 为阈值划分后两类像素占比，μ_0、μ_1 为对应类均值；有效阈值要求两类均非空。PL 二值图用于快速发现结构候选，保留的 Gray8 图像供 PS 局部重新二值化，因此前端阈值不直接决定最终解码的全部像素判断。

（4）**四方向扫描**。二维码定位图案（finder pattern，简称 finder）沿中心线具有近似 1:1:3:1:1 的黑白游程比例。硬件在水平、垂直和两条对角方向检测该特征，以整数比例比较判断游程误差并估计中心和模块尺度。多方向命中为旋转目标提供候选，行列历史放入片内存储，避免对整帧反复扫描。

（5）**候选筛选**。首先按空间位置与模块尺度聚类多方向命中，再依据三个定位点的近直角关系、边长及尺度一致性构造 ROI。依据定位图案匹配质量和几何一致性计算结构置信度评分，用于确定候选保留优先级；再结合定位点对应关系与交并比（IoU，即两区域交集面积与并集面积之比）进行去重，并对满足合并条件的重叠区域取外接范围合并，最终输出最多 30 个候选。该评分用于衡量结构匹配程度，成功解码仍由后端定位、采样及纠错确认。

（6）**数据交接**。VDMA 写入带首尾保护的 Gray8 帧，候选元数据通过独立的 ROI 元数据存储区（bank）发布。源帧号、配置和更新代次共同标识一次输入；驱动在写入完成后核对图像保护信息与候选来源，保证 ROI 坐标实际指向将被识别的同一幅图像。

（7）**HLS 透视网格优化加速器**。AXI-Lite 用于任务参数及状态访问，HP2 读取打包二值 ROI、结构特征和候选系数。输入在片内存储中复用，评分核执行后返回最优系数、分数及完成状态。整网格算法与候选树组织集中在第 2.3.2 节说明。

（8）**彩色显示**。显示 VDMA 通过三缓冲与 Genlock 协调读写，在确认存储提交后扫描完整彩色帧。输入采集与 HDMI 输出以不同节奏工作：相机约产生 30 个不同源帧每秒，HDMI 按 720p60 时序重复扫描完整帧，显示刷新率与新图像产生率分别计量[8]。

（9）**OSD 叠加**。PL 将绿色框线与透明背景文字同视频像素同步合成。PS 先将完整框文场景写入非活动标注存储区（bank），再提交切换请求；像素域在垂直消隐起点同步切换描述符和字符，并返回生效确认。确认前冻结待提交存储区，避免一幅画面混用两次更新的框与文字。

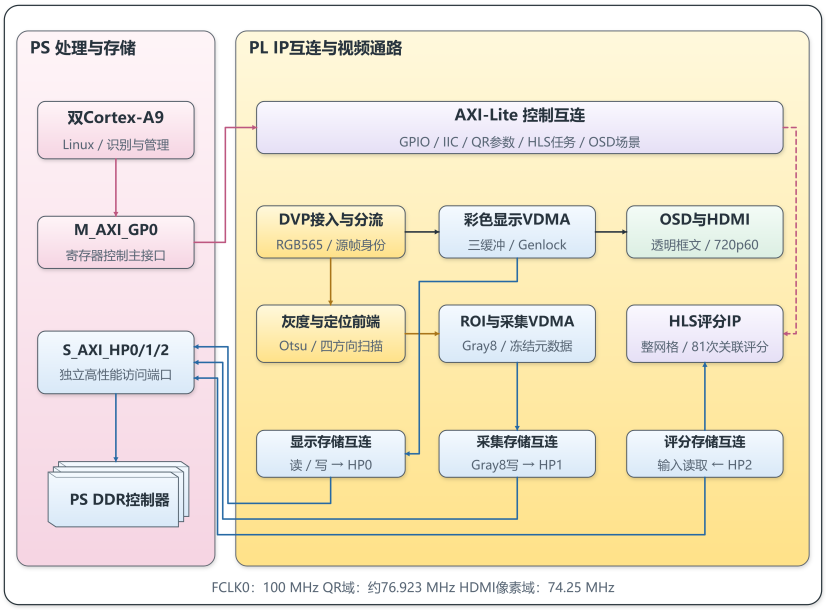


图 2-2 硬件功能框图

表 2-2 PL 外设地址映射

地址	模块	主要功能
0x41210000 / 0x41220000	相机 GPIO	reset / arm
0x41600000 / 0x41610000	AXI IIC	相机 / HDMI 配置
0x43000000 / 0x43010000	VDMA	彩色显示 / 灰度采集
0x43C00000	QR 前端	参数、状态、ROI 元数据存储区（bank）
0x43C10000	HLS 透视网格优化加速器	提交任务与读取结果（fitness）
0x43C20000 / 0x43C30000	timing 观测	采集 / 评分阶段记录
0x43C40000	OSD	场景提交、ACK 与显示状态

2.3 软件系统介绍

2.3.1 软件整体介绍

PS/Linux 端采用驱动管理、C 语言识别与 Python 调度的分层架构，承担精确解码、结果管理及软硬件协同。采集链路提前准备空闲 DMA 缓冲，将同帧灰度图像、候选区域和来源身份保存为私有快照，使下一帧采集与当前帧解码交叠，并通过缓冲所有权约束数据复用。算法侧在候选区域内重新执行 Otsu 二值化、连通域分析和精确定位，建立透视网格；预生成双精度候选系数树，调用 PL 完成评分优化，再进行模块采样、格式解析与纠错，恢复完整内容。调度侧结合候选质量、尺度及等待时间分配处理机会，以源帧和作业身份贯通输入与结果，根据内容和位置关联目标、抑制重复输出，并生成完整框文场景。其特点是将并行采集、数值一致性与结果关联融入统一处理流程。

2.3.2 软件各模块介绍

（1）**启动检查**。核对运行包、启动文件、内核模块及硬件接口能力，确认设备资源归属和参数后再启动服务。这样可以在执行采集与评分前发现固件、驱动或应用不匹配，避免把接口错误误判为算法失败。

（2）**采集与快照**。内核 DMA 三槽依次经历空闲、硬件写入、完成和用户持有状态，只有空闲槽可交给硬件写入。完成后核对首尾保护与同帧 ROI，再将 Gray8 及冻结元数据复制到用户态私有槽。物理槽归还后可接收新帧，解码期间则独占私有输入，状态见图 2-3[11]。

保留一次灰度复制，是为了分离 DMA 复用与软件处理的生命周期。解码耗时随候选数量和图像质量变化，若长期直接持有物理槽，会压缩相机可用缓冲。私有队列满时采用背压等待，既不覆盖正在读取的数据，也不通过丢弃未处理输入制造吞吐提升，从而在有限缓冲条件下兼顾数据完整性与持续处理能力。

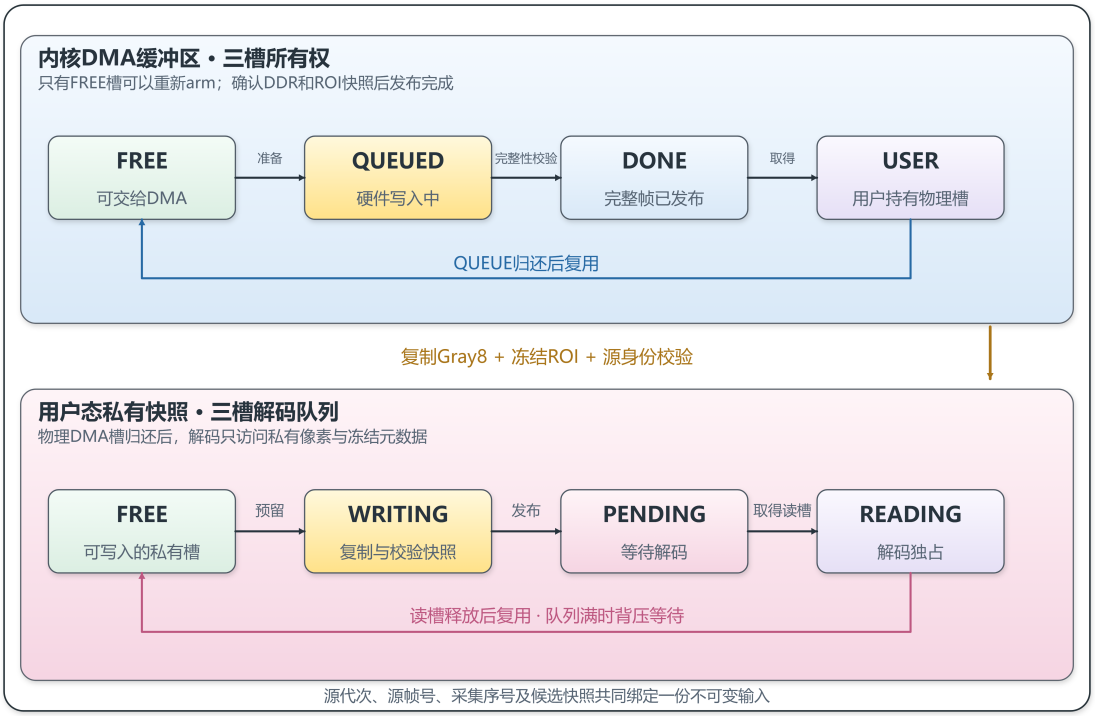


图 2-3 内核 DMA 槽与用户态私有快照所有权

（3）**候选调度**。结合质量、模块尺度、等待年龄和近期识别状态组织候选，避免同帧同区域重复解码，并给予较难候选处理机会。每开始下一 ROI 前检查 60 ms 软预算，已经开始的 ROI 完成后再交接结果。该预算约束调度节奏，不是每帧端到端耗时的硬上限。主流程见图 2-4。

（4）**局部识别**。首先裁取候选 Gray8 区域，执行局部 Otsu 二值化和连通区域分析，重新寻找定位图案并建立透视网格。由于局部区域的亮度分布不同于全帧，重新计算阈值可使后端基于自身输入判断黑白模块，不直接沿用 PL 候选扫描的二值结果。

网格优化完成后，按模块位置采样，读取格式信息，去除掩码并恢复交织数据块，再执行 Reed–Solomon 纠错和内容解析。成功结果同时返回内容与角点，局部坐标加 ROI 原点转换为原图四角[6]。候选发现、精确定位和纠错分别承担缩小范围、恢复几何及确认内容的作用。

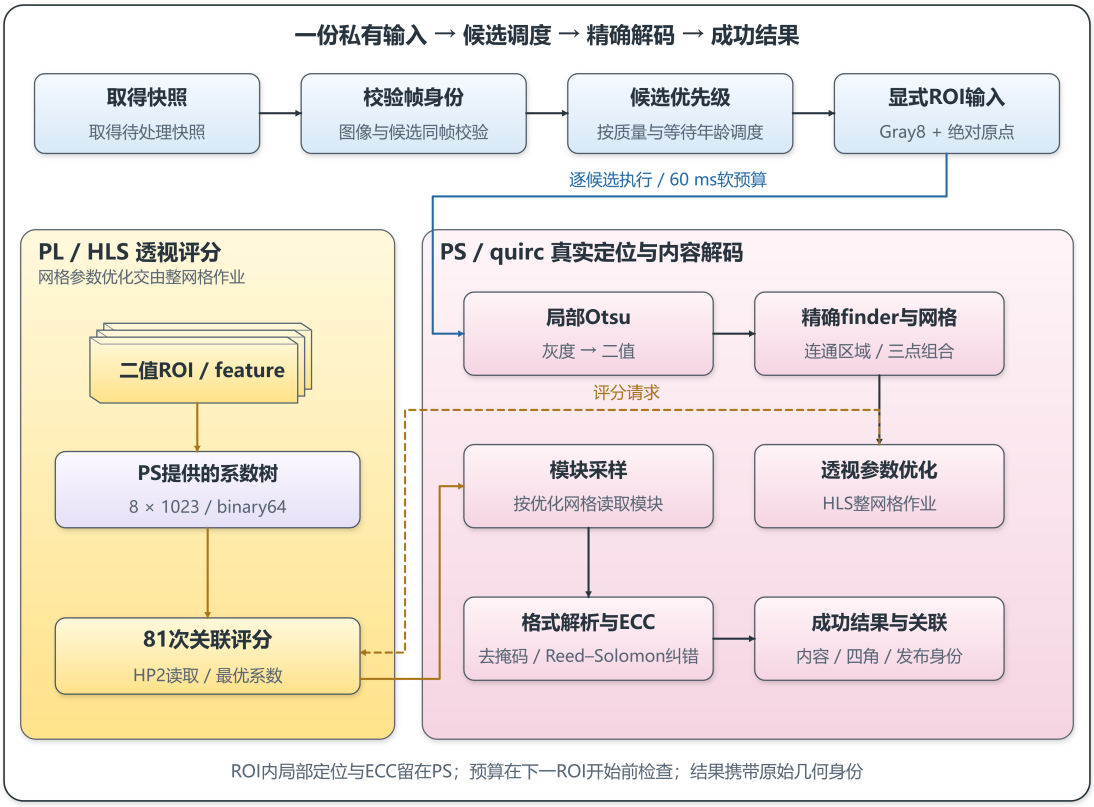


图 2-4 识别主循环与候选处理流程

（5）**透视网格优化**。本环节由 PS 与 HLS 透视网格优化加速器共同完成。二维码在旋转或倾斜拍摄时，其规则网格在图像中会发生几何变形。为确定每个黑白模块的采样位置，系统采用由八个系数描述的透视变换，将二维码规则网格中的坐标映射到图像像素位置，并通过评分调整这些系数，使采样位置与实际图案对齐。评分利用定位图案、时序图案等已知黑白结构：在每个参与评分的模块内，取 0.3、0.5、0.7 偏移的九个组合点读取二值像素，按观测值与期望值的匹配情况累计分数。二值 ROI 按位打包，并在多次评分期间复用，减少相同像素的重复传输。

透视优化先计算一次初始分数，再进行五轮、每轮八个系数的正负试探，共 $1 + 5\times8\times2 = 81$ 次评分。步长逐轮减半，仅分数严格提高时接受新系数。后续候选依赖前面的接受或拒绝结果，因此 81 次评分具有顺序关系，不能简单按独立样本重排。

PS 预生成双精度候选系数树，保存不同决策路径上所需的精确位模式；PL 根据当前评分沿接受或拒绝路径读取下一候选，保持软件浮点计算顺序。硬件在

一个作业内顺序执行相关评分与决定，内部样本循环采用 II = 2 流水；II 表示相邻样本迭代的启动间隔，并不表示两周期完成整个网格。作业组织见图 2-5。

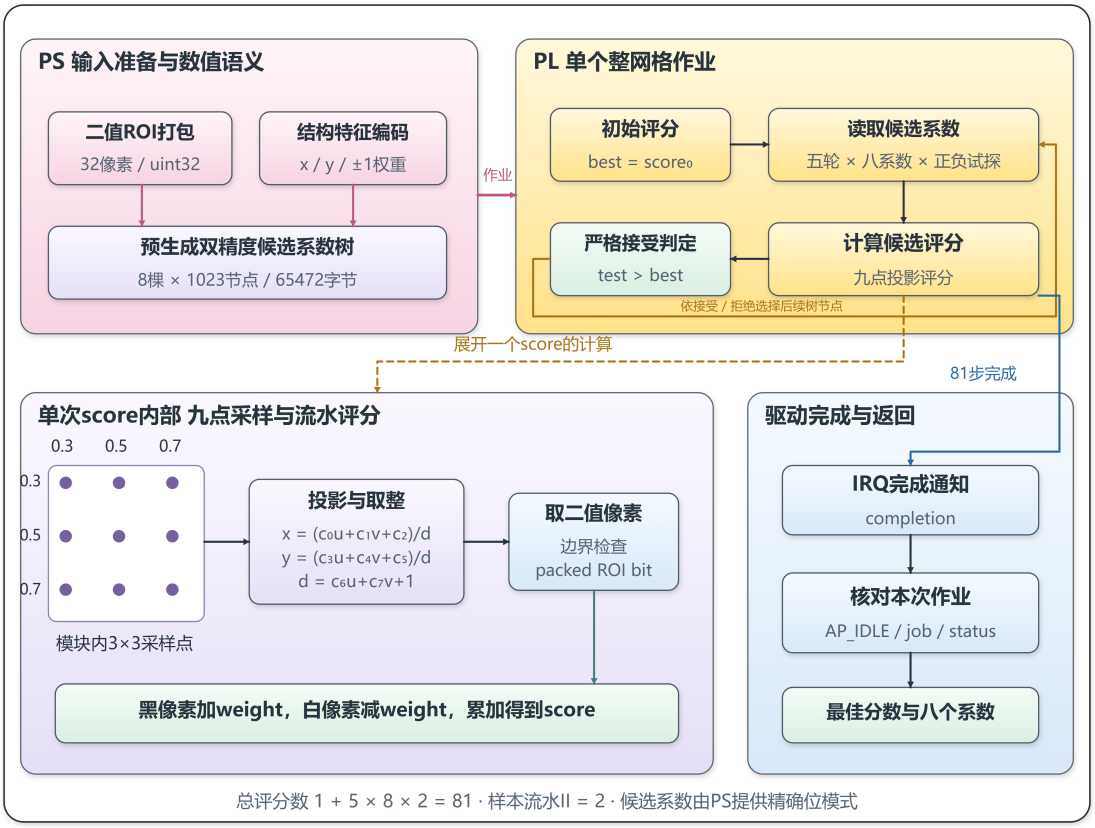


图 2-5 HLS 透视网格优化加速器的作业组织与候选系数树

HLS 透视网格优化加速器的硬件连接见图 2-6。图中保留 Vivado 实际实例名，fitness 负责整网格评分与接受决策；fitness_memory 将评分核的 DDR 读请求接入 HP2 端口；控制接口接收 PS 提交的任务参数，完成后返回中断。fitness_reset 与复位逻辑共同控制评分核复位，fitness_timing 仅被动观测控制与完成信号，不参与评分决策。

（6）**完成与恢复**。每次提交评分任务时分配任务编号，并记录其对应的 ROI 和来源帧。PL 完成处理后返回完成编号及状态，PS 侧驱动收到中断通知后，核对硬件是否已结束执行、完成编号是否与当前请求一致，再接收评分结果，避免误用旧任务的返回值。输入超出支持范围时从原始系数回退软件评分，并记录回退开销。超时处理先确认总线读取是否结束；无法确认时隔离设备并保留相关内存，防止异步读取已释放缓冲。

（7）**结果关联与重复输出抑制**。解码成功后，将完整内容、四角坐标及来源

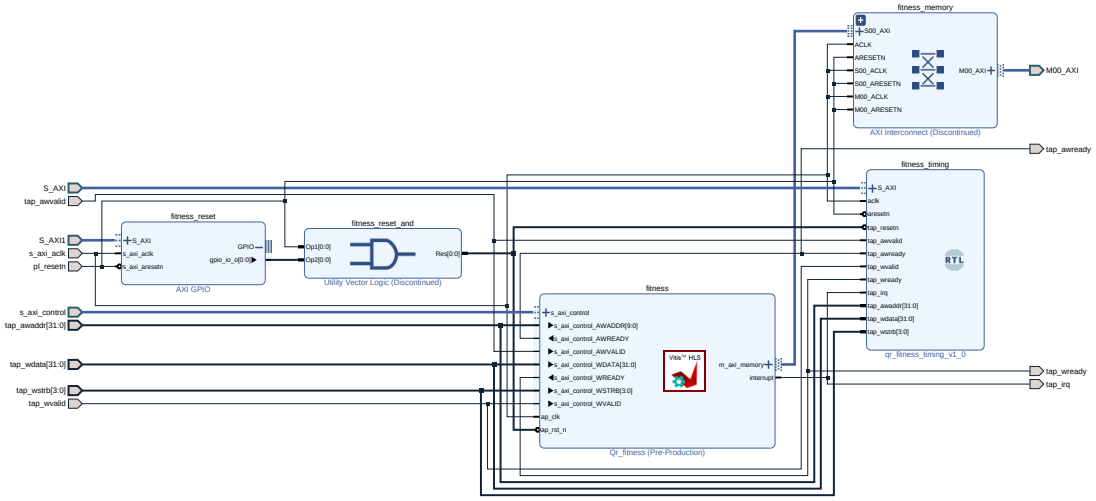


图 2-6 HLS 透视网格优化加速器及其访存、复位连接

帧一起保存，并结合目标位置、尺寸和运动连续性，判断新结果是否属于已经记录的目标。系统设置输出记录的保留时间，当前实现为自最近一次匹配起 6 s；保留期内再次读到内容一致且空间位置或运动关系匹配的目标时，更新其位置与时间，不重复新增输出记录。若相同内容位于可区分的不同目标上，则分别记录；已发布记录超过保留时间仍未匹配时失效，后续再次出现可作为新记录处理。逐帧成功解码次数、去重后的输出记录数，以及实验中的唯一内容命中数分别统计。

（8）**显示布局**。以成功解码四角生成外接框，短标签就近依附所属目标，并避开画面边界、HUD 和其他标签。几何来源不明、过期或关联存在歧义时隐藏标注，无法可靠放置的标签也不强行显示。

（9）**停止释放**。先关闭新帧准入，等待在途采集与评分结束，再同步 IRQ、终止 DMA 并解除映射；用户态先结束采集线程，再释放私有输入。识别停止时清除标注。该顺序使设备不再访问内存之后才回收资源，保证启停与正常运行使用一致的所有权规则。模块接口汇总见表 2-3。

表 2-3 处理模块输入输出及作用

处理模块	关键输入	输出与作用
启动核验	运行包及设备状态	确认文件、内核和接口匹配
采集与快照	Gray8、ROI 及源帧身份	生成稳定的同帧输入
候选调度	质量、尺度与等待状态	确定 ROI 处理顺序
局部定位	候选灰度图及原点	得到定位图案与初始网格

处理模块	关键输入	输出与作用
HLS 透视网格优化加速器	二值 ROI、结构特征及系数树	执行整网格评分与接受决策，返回优化后的透视系数
采样与纠错	优化网格及模块数据	解析完整内容和四角
结果关联	内容、位置、时间及来源	关联目标并抑制重复输出
场景提交	目标框位置、标签与状态	同步更新框线及文字

2.4 大模型协作与技能包提炼

2.4.1 协作方式与记录

项目使用 ChatGPT/Codex 辅助需求分析、源码阅读、方案比较、代码编写、工具反馈处理和实验整理。模型工作于开发端，板端实时识别采用结构定位与纠错算法；团队负责硬件连接、现场操作、测试条件确认和方案采用。协作产物包括工程实现、验证记录和可复用技能。

协作按“目标与约束—可验证方案—执行验证—反馈纠错”组织。每次修改说明适用输入和预期证据，模型建议经工具结果或实验反馈检验后再决定采用；出现反例时保留原基线并修正判断。交互与案例记录随项目资料保存，下述案例将原始摘录与工程归纳分别标明。

2.4.2 代表性提示与纠错案例

案例：性能优化与软硬件架构调整。本案例按团队提出任务、模型分析建议、实测验证和反馈纠错的交互过程展开；带引号的内容为交互摘录，其余为工程归纳。

团队（提出任务）：完成裸机项目 C04 开发后，团队发现采集吞吐难以进一步提升，于是保留 C04 冻结基线，转向板端 Linux 系统并部署已有基线，要求模型通过分段计时分析性能瓶颈，确定耗时最长的处理环节。

模型（分析与建议）：模型识别出透视网格迭代评分是主要计算热点，同时发现采集环节的帧接收间隔偏长。据此提出提前 re-arm 优化采集流程，并采用 HLS 流水线及寄存器缓存优化评分模块。

实测验证：相机原始帧间隔约为 33.33 ms，原接收间隔约为 66.67 ms；提前 re-arm 后，采集吞吐率由约 15.07 FPS 提升至 29.94 FPS，完整评分阶段获得约

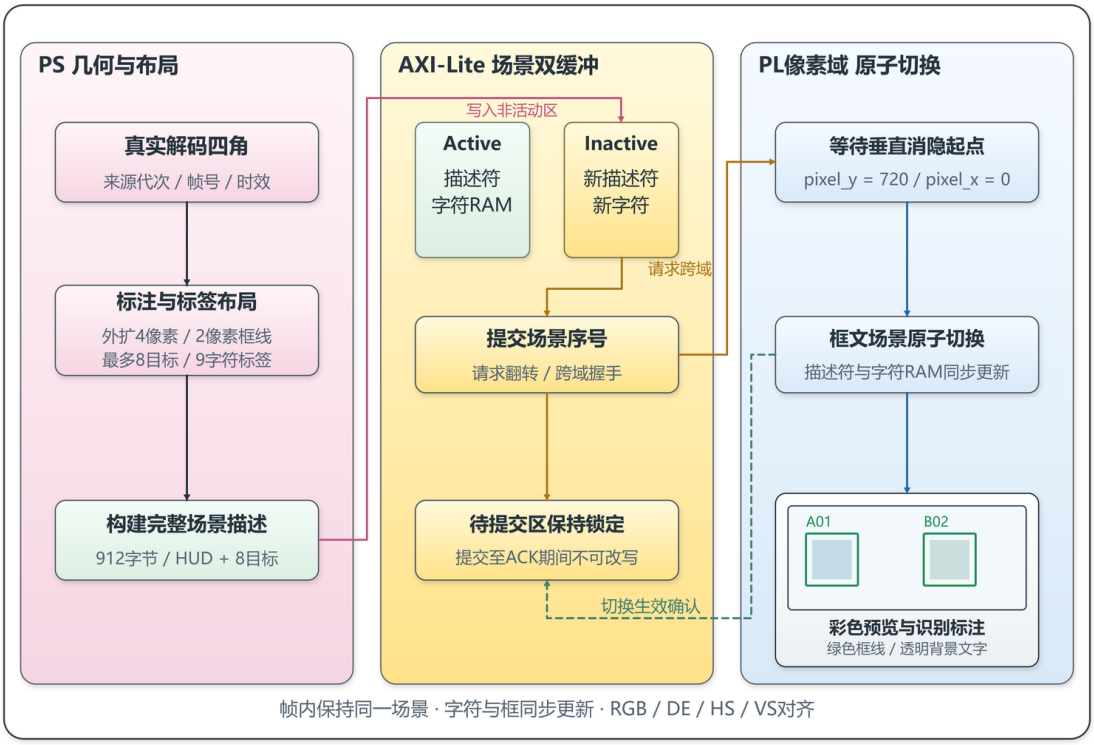


图 2-7 场景双缓冲提交流程

3.25～3.32 倍加速。相关结果见第 3.3.2 节。采集吞吐与评分阶段加速分别统计，作为后续比较的依据。

模型（性能汇报摘录）：“11.43 FPS 提升至 13.87 FPS，提升约 21.4%”。**团队（追问摘录）：**“之前 C04 已经到达约 14 FPS，为什么现在测试反而只有 11？”团队以既有测试结果提出质疑，要求核对性能比较的基线与条件。

模型（复核并修正解释）：经核对，模型承认原回答的性能参照口径不够清晰，进一步区分历史网页测试 14.76 FPS、短时固定纸片测试、重复固定纸片测试以及 HDMI 显示负载等条件。评分网格数量由每帧 2.29 个增加至 2.98 个，可以解释部分性能差异，但未被认定为唯一原因。

团队（复核架构）：团队进一步指出架构图中灰度转换职能的分配存在疑问。**模型（修正图示）：**模型检查后承认图示错误，明确该测试阶段由 PL 执行摄像头图像的 RGB565 到 Gray8 转换，PS 当时执行的是显示所需的 Gray8 到 RGB565 格式转换，并据此修正架构图。

团队（验收与取舍）：团队保留 C04 冻结基线，在统一场景与输入条件下，通过分段计时、SOF 帧间隔观测和完整识别统计核验模型提出的优化。某次对

照中，最新版虽达到 25.12 FPS，却仅成功识别 72/100 个目标，低于原 C04 的 97/100，因此未被认定为整体优化成功；后续复测恢复至 95/100，仍保留首轮失败记录。II=4 方案虽通过前期 HLS 检查，也因整机最终时序不满足要求而被淘汰。

交互归纳：团队负责确定目标、提供实测反馈并作出采用决策，模型负责分析瓶颈、提出方案并依据反例修正解释与实现。性能比较需明确基线、输入和测试条件，架构图需准确反映 PL/PS 实际职责，优化结果需同时接受完整功能与整机时序检验。

2.4.3 技能的提炼与复用

项目形成 13 个现行技能包，分为 3 个通用包和 10 个专项包，技能名称及主要功能见表 2-4。

提炼以真实失败或观测为起点，将固定板卡参数与通用判断步骤分离，整理为适用问题、输入条件、执行步骤、验证产物和失效边界。例如，快速失败实验转化为性能口径核验，图像错帧转化为缓冲所有权检查，硬件与驱动不匹配转化为接口和交付核对。表 2-4 同时概括了各技能的经验来源或使用边界。

使用者先按问题选择相应技能，提供目标工程的接口、日志或冻结输入，再按步骤取得证据。技能可以复用判断方法，具体地址、时钟和版本仍需在新工程确认；协议、数值范围或硬件条件变化时，重新核验相应结论，不能直接套用本项目的历史倍率与参数。

表 2-4 13 个现行技能的中英文名称及功能

技能名称	主要功能	经验来源及边界
系统化调试 systematic-debugging	核实观测、定位首个失效边界，检验竞争假设。	来源于分层排错；依赖实际日志及可观察输入。
性能瓶颈分析 performance-bottleneck-analysis	统一指标、负载和窗口，按阶段确认优化收益。	快速失败曾造成虚假提速；同时验收正确性。
AI 辅助工程 ai-assisted-engineering	组织需求、模型协作、验证和交接。	来源于协作纠错；建议需由实际情况分析检验。
硬件构建与交接 hardware-build-and-handoff	绑定源码、硬件接口及运行产物，组织验收。	应对软硬件混装；构建与目标运行分别核验。
Vivado 与 Vitis 流程 vivado-vitis-flow	组织 RTL/HLS IP 生成、集成、导出及软件构建。	流水候选曾出现时序失败；按实际工具重验。
流数据通路验证 streaming-data-path-verification	检查接受拍、背压、打包和帧边界。	来源于流控问题；另需核对 CDC 与物理时序。

技能名称	主要功能	经验来源及边界
共享缓冲生命周期 shared-buffer-lifecycle	约定所有权、DMA 可见性及停止释放顺序。	来源于错帧和异步复用；实板核验可见性。
PS-PL 接口契约 ps-pl-interface-contract	联合检查地址、ABI、任务身份和完成语义。	地址相同可能语义不同；协议改变时重验。
相机与 HDMI 诊断 camera-hdmi-diagnosis	分离源控制、格式、存储、处理及显示故障。	来源于视频集成；不预设传感器或显示模式。
Linux 运行与发布 linux-runtime-and-release	核对目标 ABI、驱动、服务及部署恢复。	来源于运行环境匹配；内核变化需重核模块。
网络流诊断 network-streaming-diagnosis	定位链路、帧重组、传输及过载问题。	来源于网络回放；保留完整帧与结果身份。
结果关联与发布 result-association-and-publication	管理内容、空间实体、队列和可见输出关联。	来源于错位和过期结果；与解码失败分开诊断。
算法移植与回放 algorithm-porting-and-replay	冻结输入，逐阶段比较数值及语义。	来源于平台差异；新数值范围及物理输入另验。

验证分为入口格式与引用检查、文件清单核对、新场景内容试用和部分校验资源的实际执行。其中相关资源已完成 52 项辅助检查、XSim 正例与四类故障注入，以及 59 个冻结正例和 3 组模块比较。故障注入用于确认检查器能够识别所设计的错误。

第三部分　完成情况及性能参数

3.1 整体介绍

系统已完成相机采集、独立彩色预览、动态多目标识别、HLS 透视网格优化加速及 HDMI 标注，形成可运行平台，实物平台见图 1-1。运行管理完成三轮启停、系统重启保持停止及重启后启停检查。主机通过以太网建立 SSH 连接，或通过 UART 串口登录板端，进行现场控制，查看识别结果与日志。

3.2 工程成果

3.2.1 软件成果

软件交付包括三个内核驱动模块、C 识别库、Python 应用、systemd 服务及安装恢复脚本。配套清单和日志用于查询运行身份、设备状态和处理进度，安装恢复入口支持还原本次部署变更，形成从构建产物到板端操作的完整交付。

3.3 优化过程与资源对比

3.3.1 图像采集等待的定位与优化

为区分图像采集与二维码计算对处理速度的影响，团队先进行仅采集测试，即接收完整图像并统计帧率，暂不执行二维码定位与解码。相机每秒约输出 30 帧，但软件实际取得完整图像的速度只有约 15.04 FPS，说明采集环节本身存在等待。进一步记录帧开始、图像写入 DDR 完成、中断通知及下一帧接收准备的时间，发现驱动等待上一帧的候选区域等元数据时，尚未为下一帧准备好接收缓冲区。由于硬件必须从帧首接收完整图像，错过帧首后只能等待再下一帧，因此出现每两帧只接收一帧的现象。

针对这一问题，驱动在存在空闲缓冲区时提前准备下一帧接收，使接收准备与上一帧元数据的读取重叠进行，减少因准备过晚而错过帧首的情况。同时，已经完成采集或正在被软件读取的缓冲区仍保持保护，避免新帧覆盖已有数据。修改前后均进行三轮、每轮 300 帧的仅采集测试，优化后的接收帧率达到约 29.92 FPS，接近相机的源帧率。由于测试未执行二维码解码，这项提升反映的是采集等待的减少。缓冲区状态与快照机制见第 2.3.2 节。

3.3.2 从单次评分加速到整网格协同

在取得图像后，识别程序需要确定二维码各黑白模块的采样位置。对于旋转或倾斜拍摄的二维码，程序通过调整透视变换系数，使规则网格与图像中的实际图案对齐。每次调整都需要进行投影、像素采样和评分，再根据分数决定是否接受新系数；一次网格优化共进行 81 次相互关联的评分。分段计时表明，重复评分是 PS 端的主要耗时环节，因此团队优先将这部分计算交给 PL 加速。

初版通过 HLS 生成评分 IP，由 PL 执行单次评分中的投影、采样和分数累加，PS 负责比较分数并决定是否更新系数。此时，PS 仍需逐次提交评分任务，等待 PL 完成后读取分数，再准备下一次评分。虽然硬件缩短了计算时间，多次任务提交和结果读取仍会产生开销，因此这一阶段还减少了未变化参数的重复寄存器写入。

在评分 IP 内部，团队采用流水线重叠执行不同样本的计算，并比较实际流水间隔、资源占用和布局布线时序，最终选择 II ＝ 2，即每隔两个时钟周期可启

动下一次样本计算。作为对照，II ＝ 4 方案的建立时间裕量为 −0.411 ns，未满足时序要求，因此未被采用。

在上述单次评分加速阶段，三批使用相同像素输入的对照试验中，完整评分阶段的耗时由 32.193～32.233 ms 降至 9.706～9.900 ms，达到原方案约 3.25～3.32 倍的处理速度。计时包含输入准备、任务控制和结果处理；累计比较 4410 个网格、357210 次评分，分数及是否接受新系数的判断均与软件参考一致，说明该阶段在保持计算结果的同时降低了耗时。

为进一步减少逐次评分的交互开销，后续方案将一个网格的 81 次关联评分和接受决策合并为一次硬件任务。PS 预先生成候选系数，PL 按照前一次评分的判断结果选择后续候选，完成整轮网格优化后再返回结果，减少 PS 与 PL 之间的反复提交和等待。同时，PS 使用 NEON 指令加快二值像素的按位打包，并在同一二值化结果仍有效时复用 ROI 输入，减少重复准备和传输。整网格任务的执行顺序与候选系数组织见第 2.3.2 节。

3.3.3 独立显示优化

PS 承担彩色图像复制及绘制时，会与识别争用处理器和内存资源，因此将预览改为 PL 独立通路，识别恢复为 Gray8。固定场景配对中，识别处理帧率由 16.642 提升至 20.084 FPS，约提高 20.68%。该结果说明解除显示负担能为识别释放资源，而非只改善显示观感。

独立预览约保持 30 个不同源帧每秒，显示不等待解码。该项比较对应固定场景及其自身版本条件，最终系统相对纯 PS 的整体收益另由第 3.4.3 节配对实验验证。

3.3.4 分项性能与资源成本

图 3-1 分别展示采集吞吐、完整评分阶段、PL 单次评分和独立显示的优化结果，并标注对应数值与测试条件。各面板使用自身单位和测量边界，完整评分保留三批观测范围；整机纯 PS 配对另见第 3.4.3 节。

图 3-2 比较 R2 与 R4 的资源利用率及时序裕量。独立显示与 OSD 等功能使 LUT 和 BRAM 占用增加，DSP 利用率接近原水平；资源投入与显示解耦及系统功能完善相对应。

最终实现使用 45236/53200 个 LUT、45604/106400 个 Slice 寄存器（42.86%），占用 13250/13300 个 Slice（99.62%），DSP 为 99 个，TNS 为 0。较高的 Slice 占用反映布局资源较紧；采用 BRAM 驻留输入与单套评分流水复用控制硬件成本，同时保持双精度与顺序依赖。资源数量、利用率与时序裕量均取全芯片布局布线报告。

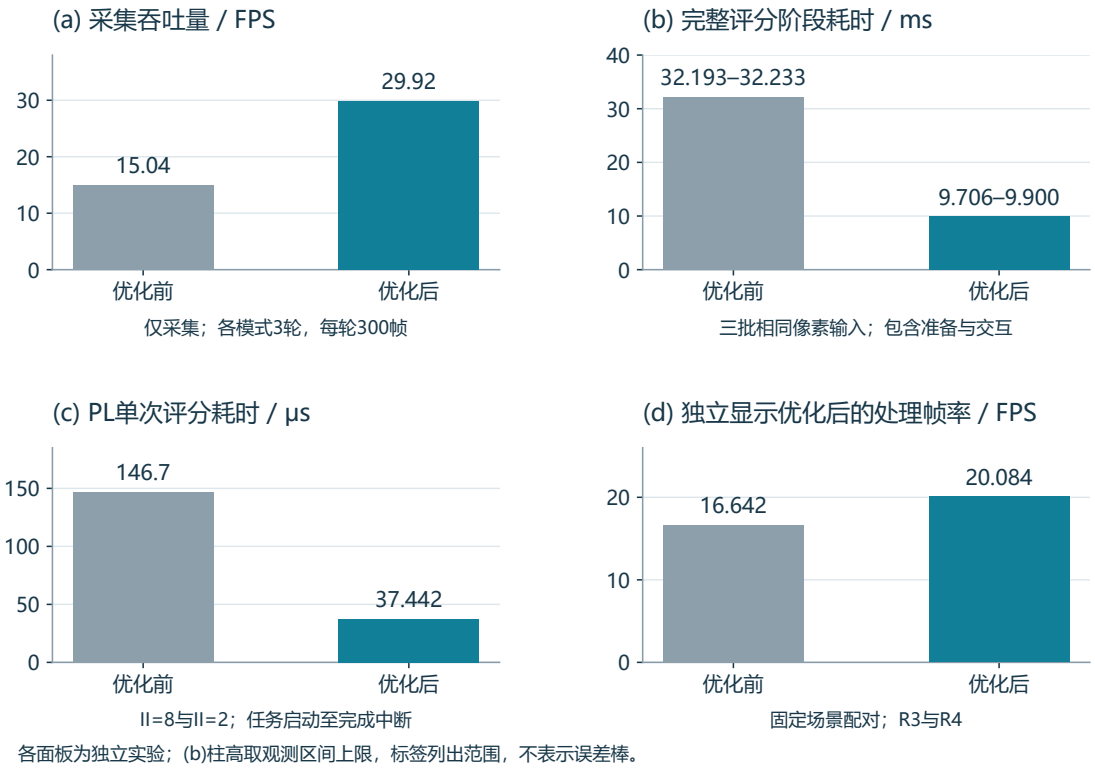


图 3-1 分项优化前后的性能变化

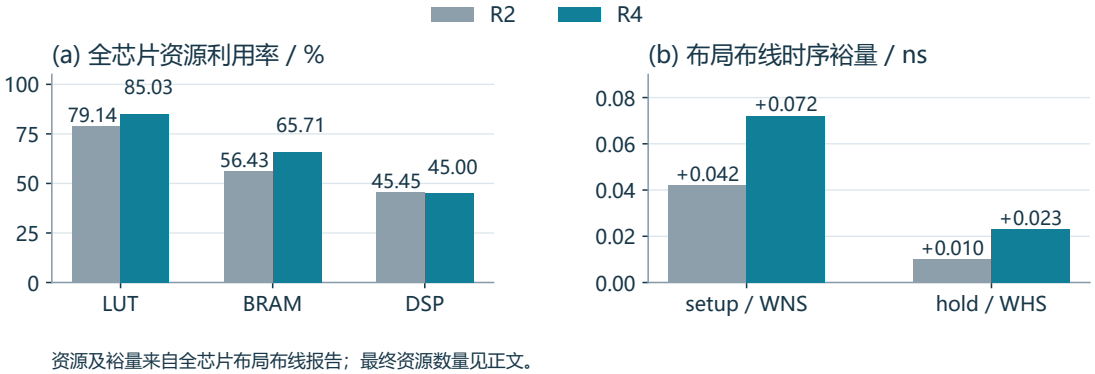


图 3-2 R2 与 R4 的资源利用率及时序裕量对比

3.3.5 同类平台识别速度对比

在前述分项性能和资源分析的基础上，为评估项目的识别处理性能，我们与具有代表性的相关项目进行数据对比，统一使用处理帧率 FPS 表示各资料的速度数据。表 3-1 汇总各资料直接给出的帧率，以及由读码吞吐量或单张处理耗时换算的等效处理速率；图 3-3 给出对应对比。

表 3-1 同类平台识别速度对比（等效处理速率）

方法	平台	识别速度（FPS）	原始测试数据	数据类型
PALLAS 65[13]	Zynq UltraScale+ MPSoC	≈ 56	56 次/s；640×480	读码吞吐量换算
本项目	**AX7021B / Zynq-7020**	**27.939**	27.939 FPS；1280×720	原文直接给出帧率
Gu 等（2024）[4]	Xilinx ZYNQ	≈ 10.75	93 ms/张	单张处理耗时换算
Zynq + ZBar（2019）[14]	Zynq-7000 SoC	> 2	连续 100 次实验，平均耗时 < 500 ms/次	平均处理耗时换算

换算说明：处理耗时统一按 $FPS_{\text{equiv}} = 1000/T_{\text{ms}}$ 换算，其中 T_{ms} 为以毫秒计的处理耗时；原文直接提供的帧率保留原值。PALLAS 65 的 56 次/s 按等效处理速率约 56 FPS 表示；Gu 等方案为 $1000/93 \approx 10.75$ FPS；Zynq + ZBar 方案由 $T_{\text{ms}} < 500$ 换算得 $FPS_{\text{equiv}} > 2$。

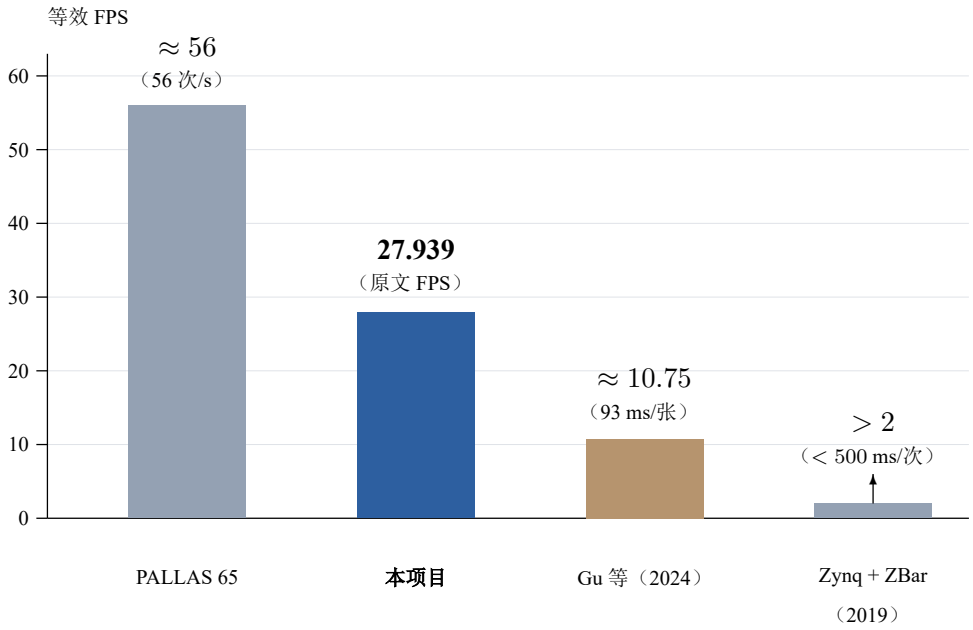


图 3-3 同类 Zynq 平台二维码识别速度对比

PALLAS 65 的等效处理速度约为 56 FPS[13]，Gu 等（2024）方案由 93 ms/张换算得到约 10.75 FPS[4]，Zynq + ZBar（2019）方案由平均耗时小于 500 ms/次换

算得到大于 2 FPS[14]。本项目的平均识别处理速度达到 27.939 FPS，按上述数值比较，约为 Gu 等方案的 2.60 倍。

从部署平台看，PALLAS 65 采用 Zynq UltraScale+ MPSoC，配置四核 Cortex-A53，主频为 1.2 GHz[13]；本项目则采用处理器配置相对较低的 AX7021B（Zynq-7020），处理器为双核 Cortex-A9[15]。通过 PS 与 PL 协同设计，本项目在较低处理器配置的平台上实现了接近 30 FPS 的处理速度，兼顾了高速识别与较低硬件配置需求。[2]

3.4 性能测试与加速效果验证

3.4.1 测试场景与统计方法

为验证系统对运动二维码的识别能力，团队分别开展虚拟网页仿真测试和真实传送带测试。前者便于重复设置二维码的尺寸、旋转角度和运动速度，用于统计不同条件下的内容读取能力，并比较 PL–PS 协同方案与纯 PS 方案的处理性能；后者使用实际传送带输送二维码，验证系统在实物场景中的读取能力。两类测试均已完成，结果分别统计：第 3.4.2 至第 3.4.3 节报告虚拟网页仿真测试，第 3.4.4 节报告真实传送带测试。

在虚拟网页仿真测试中，屏幕连续显示运动二维码，由 OV5640 摄像头拍摄后送入板端处理，使测试覆盖屏幕显示、光学成像、图像采集和内容解码的完整过程。实验固定屏幕显示与相机取景条件，设置 K4、K5、K6(K 表示源画布像素/模块) 三档二维码尺寸及五档运动速度，并改变二维码旋转角度。协同方案共测试 50 组，每组包含 100 个不同完整内容，累计 5000 个目标；每档速度对应 10 组、共 1000 个目标。虚拟网页仿真测试配置见表 3-2。

为评估 PL 加速带来的系统收益，首轮 10 组同时设置纯 PS 对照，协同方案与纯 PS 方案各测试 1000 个目标，每档速度各 200 个。两种方案使用相同的目标内容、尺寸、旋转角度、车道及形状配置，以便比较内容命中与识别处理帧率；运行顺序为先协同方案、后纯 PS 方案，本次测试未进行交错随机化。其余 40 组仅测试协同方案，用于追加统计不同条件下的读取结果。

[2]**数据来源：**PALLAS 65 的平台配置及 640×480 下 56 次/s 的数据取自产品资料《大恒图像智能产品》第 3–4 页[13]；Gu 等方案的 93 ms 单张处理时间取自其 2024 年发表于 IEEE MultiMedia 的论文[4]；Zynq + ZBar 方案的数据取自专利 CN109993018A 中连续 100 次实验、平均识别时间小于 0.5 s 的记录[14]；本项目平均处理速度 27.939 FPS 取自项目技术总结第 17.3、17.6 节；Zynq-7020 的双核 Cortex-A9 配置见 AMD 产品资料[15]。

统计以每个目标的展示过程为单位，通过组号和预设完整内容确定目标。在目标展示期间，只要至少一次解码内容与预设内容完全一致，就将该目标计为命中；同一目标在连续帧中被多次正确解码，或产生重复输出，均只计一次。目标级内容召回率按命中目标数与展示目标总数之比计算；与预设内容不一致的解码结果另行记录，用于检查错误输出。

表 3-2 虚拟网页仿真测试配置与统计方法

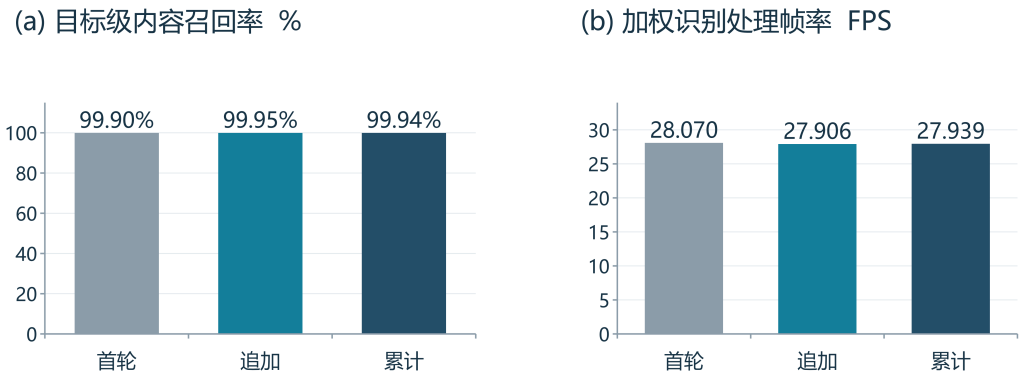
项目	测试配置
数据规模	协同方案 50 组 ×100 个，共 5000 个不同完整内容
配对对照	首轮协同方案与纯 PS 各 10 组 ×100 个
相机与源画布	相机 1280×720；源画布 1280×760
显示几何	副屏 1920×1080 全屏；DPR=1
目标尺寸	源 K4、K5、K6，K 表示源画布像素/模块
角度与车道	36 点 −180° 至 180° 序列逐组打乱并循环；双车道、密度参数 6
五档源画布速度	243、324、486、648、810 px/s；对应网页参数 1.8、2.4、3.6、4.8、6.0
每速度样本	协同方案 10 组 1000 个；纯 PS 首轮 2 组 200 个
解码设置	HLS 透视网格优化加速器（pl-grid）；60 ms 软预算；背压队列；保留 HDMI 与成功事件记录
扰动与等待	网页照度控件 100%；无模拟模糊、污损或曲面；5 s 前导、2 s 尾部等待

本轮 5000 个目标及首轮 1000 个对比均使用 11.2 英寸、3200×2136 显示屏，网页速度参数 s 按 $v_{\text{source}} = 135s$ 更新源画布水平坐标，因此本报告的速度分项统一采用源画布 px/s，表示网页画布中的运动速度。

3.4.2 累计结果与速度分项

如图 3-4 所示，首轮 1000 个二维码中成功读取 999 个，随后追加 4000 个二维码，成功读取 3998 个。两批测试累计 50 组、共 5000 个不同二维码，系统成功读取 4997 个完整内容，另有 3 个未能成功读取，目标级内容召回率为 99.94%。首轮与追加测试均保持较高命中，说明增加测试数量后，系统在本次尺寸、角度和速度范围内仍保持较完整的内容读取能力。将去重后的解码内容与预设内容进行比对，输出中没有出现预设内容之外的结果，按完整内容集合计算的精确率为 100%。同时，累计加权识别处理帧率为 27.939 FPS，反映了系统连续接收图像并完成识别处理时的整体吞吐水平。

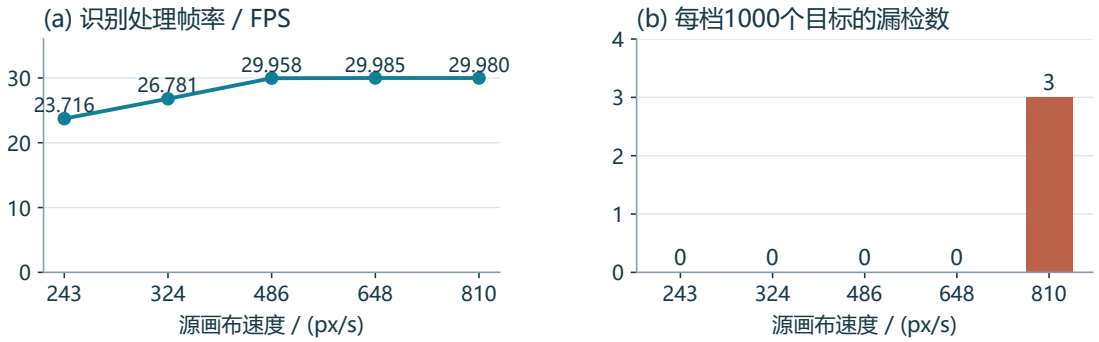
为进一步分析运动速度对读取效果的影响，将 5000 个目标按五档速度分别



命中 / 展示：首轮999/1000，追加3998/4000，累计4997/5000；漏检分别为1、2、3个。
累计为前两部分合并统计，非第三次独立实验；处理帧率按窗口时长加权。

图 3-4 首轮、追加及 5000 次累计测试结果

统计，每档包含 1000 个目标。如图 3-5 所示，最高速度档成功读取 997 个，目标级内容召回率为 99.70%；其余四档均成功读取全部 1000 个目标。各档识别处理帧率为 23.716～29.985 FPS。随着目标运动速度提高，其在相机视场内的停留时间缩短，对系统完成候选提取、定位和解码的及时性提出更高要求。在本次源画布速度 243～810 px/s 的测试范围内，系统各档目标级内容召回率均不低于 99.70%，表明系统在连续处理运动图像的同时，能够保持较高的二维码内容读取完整性。



虚拟网页仿真测试；1920×1080副屏；网页参数1.8、2.4、3.6、4.8、6.0。
810 px/s档命中997/1000，其余各档1000/1000；本图不表示真实传送带带速。

图 3-5 5000 次累计测试的速度分项结果

处理 FPS 由累计处理帧数差与时间差计算，包含等待，并按窗口时长加权，而非对各组帧率简单平均。不同速度下候选负载和有效处理窗口不同，帧率与内容召回需结合分析。独立预览按不同源帧计数，合并约 30.001 FPS；HDMI 刷新为 60 Hz，三种口径分别统计。

3.4.3 首轮 1000 次的纯 PS 配对对照

为验证 PL 对系统的实际加速作用，首轮对 PS、PL 协同方案和独立纯 PS 识别基线进行了对比。纯 PS 在 ARM 端完成灰度、阈值、候选、ROI、评分及解码，PL 保留相机传输与显示；协同方案将前端和评分交给 PL。

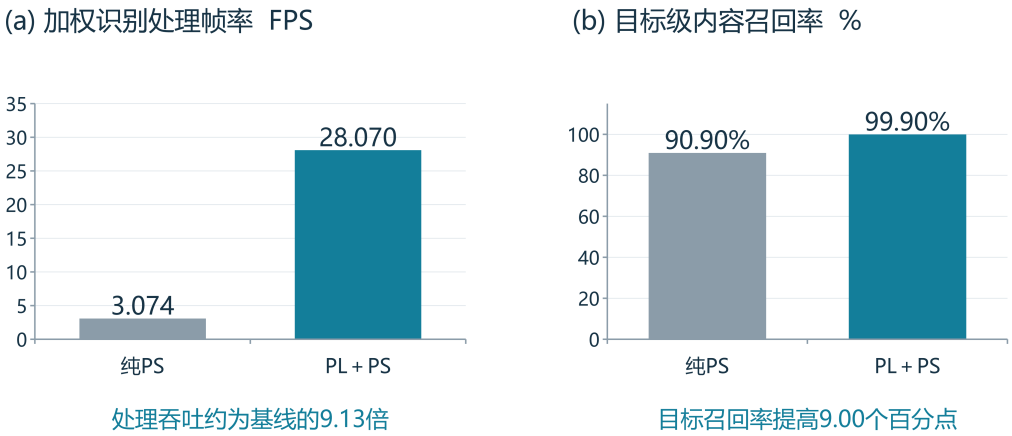


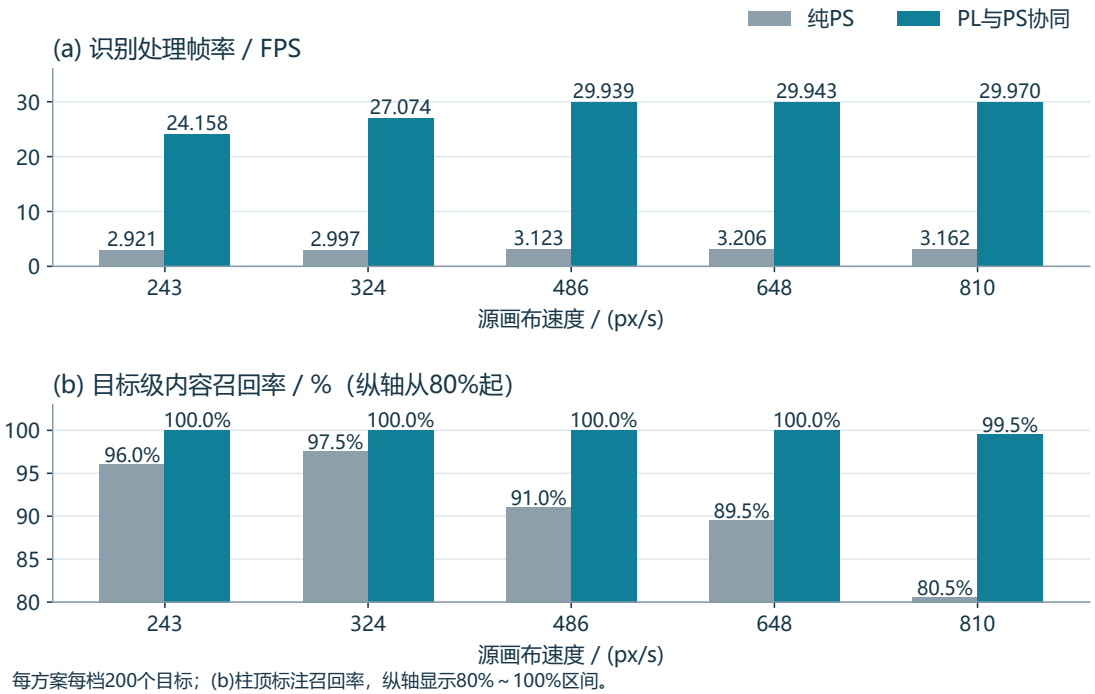
图 3-6 首轮 1000 次配对的整体加速与召回收益

吞吐方面，引入 PL 后加权识别处理帧率由 3.074 提升至 28.070 FPS，达到基线的约 9.13 倍。该提升覆盖实际输入等待、算法执行与协同开销，表明局部硬件加速已转化为整条识别链的有效处理能力。

读取完整性方面，命中由 909/1000 增加至 999/1000，目标召回率由 90.90% 提高至 99.90%，增加 9.00 个百分点；漏检由 91 个降至 1 个，减少约 98.90%。说明协同方案在提高速度的同时，也提高了动态目标被正确读取的比例。

图 3-7 中五档速度下吞吐均提升，倍率为 8.27～9.59。在最高源画布速度 810 px/s（网页参数 6.0）下，协同方案为 29.970 FPS、命中 199/200，纯 PS 为 3.162 FPS、命中 161/200，表明高速条件下吞吐与内容读取完整性同时改善。更高处理能力使有限驻留时间内拥有更多有效解码机会，是该结果的合理解释。

两方案预览均约 30 FPS，识别处理帧率的对比反映前端、评分及交互开销共同作用下的系统收益。



每方案每档200个目标；(b)柱顶标注召回率，纵轴显示80%～100%区间。

虚拟网页仿真测试；1920×1080副屏；网页参数1.8、2.4、3.6、4.8、6.0。

图 3-7 首轮 1000 次配对的速度分项对照

3.4.4 真实传送带测试

为验证系统在实际输送场景中的识别能力，使用图 1-1 所示平台开展真实传送带测试。测试二维码按 K4、K5、K6 三档尺寸分组，每组 50 个，三组均在相同传送带速度下运行，共测试 150 个目标。摄像头采集传送带上的二维码，由板端完成识别并输出内容；以每个测试目标是否被正确读取进行统计，同一目标在连续帧中多次解码只计一次。测试现场及识别显示见图 3-8。

表 3-3 真实传送带测试结果

尺寸分组	测试目标数	正确读取数	识别成功率
K4	50	50	100%
K5	50	50	100%
K6	50	50	100%
合计	150	150	100%

如表 3-3 所示，三个尺寸分组均完成全部目标的正确读取，总体识别成功率为 100%。本节成功率按正确读取目标数与测试目标数之比计算，表明系统能够在本次固定带速和三档尺寸条件下完成实际输送目标的内容读取。这 150 个目标与虚拟网页仿真测试的 5000 个目标分别统计。



图 3-8 真实传送带测试现场及 HDMI 识别显示

第四部分　总结

4.1 可扩展之处

系统已能够输出二维码内容及其在图像中的位置，后续可在此基础上增加与仓储管理系统的对接功能，用于货物信息查询、识别记录保存和编码追溯。针对不同应用场景，可以调整前端检测参数和候选区域的处理顺序。当同一画面中的二维码数量增多、尺寸减小或运动速度提高时，需要重新测试内容召回率与识别处理帧率，再确定相应的参数和软硬件配置。

4.2 不足之处

目前使用的 OV5640 摄像头模组需要手动调焦，系统尚未实现自动调焦功能。部署时，需要根据摄像头与目标之间的距离人工调整镜头，使二维码图像保持清晰。当目标距离发生较大变化时，原有对焦设置可能无法保持图像清晰，进而影响二维码定位与解码。因此，当前系统更适用于摄像头安装位置和目标运动平面相对固定的场景。后续可增加电动调焦与图像清晰度反馈控制，提高系统对目标距离变化的适应能力。

4.3 心得体会

调试采集功能时，我们发现，相机正常输出图像，并不代表软件能够及时接

收到每一帧。为了找出原因，我们先暂停二维码解码，只观察图像接收过程，再对照每帧开始和缓冲区准备的时间。检查后才发现，驱动还在等待上一帧的元数据时，下一帧已经开始了。这个问题解决后，我们在排查其他故障时也逐渐形成了习惯：先把相关环节单独运行，确认问题出现在哪里，再修改对应代码，减少反复尝试。

评分加速的过程也让我们对软硬件协作有了更直接的理解。最初，我们将单次评分计算交给 PL，但 PS 每提交一次任务，都要等待并读取一次结果，这些交互仍然占用时间。后来将一个网格的关联评分合并提交，才进一步减少了等待。通过这次调整，我们意识到，设计加速模块时，需要一起考虑输入怎样准备、任务怎样提交、结果怎样返回。只有这些环节配合好，硬件计算速度的提升才能体现在系统的实际运行中。

在大模型辅助开发过程中，我们逐渐学会把模型给出的建议放回实际工程中验证。例如，模型曾提出一项候选过滤规则，我们用已经能够成功识别的样本检查后，发现其中部分正确目标也会被排除，因此没有启用这项规则。此后，我们会先保存能够正常运行的版本，每次集中修改一个问题，再对照原版本检查结果和耗时。大模型帮助我们阅读代码、整理思路和实现方案，实际测试则帮助我们判断修改是否有效。我们也将这些排查步骤和容易出错的地方记录下来，方便后续修改时查阅和复用。

第五部分　参考文献

[1] 陈荣军, 于永兴, 钟秀媚, 等. 基于改进 CORDIC 算法的 QR 码快速检测硬件实现方法 [J]. 电子技术应用, 2021, 47(5): 92–96. DOI:10.16157/j.issn.0258-7998.200222.

[2] LAM K K L, SUM K W. An FPGA-Friendly Algorithm for QR Code Detection[C]//2023 IEEE International Symposium on Circuits and Systems. IEEE, 2023: 1–5. DOI:10.1109/ISCAS46773.2023.10181737.

[3] 方园园. 高速环境下的二维码采集与识别系统设计 [D]. 西安: 西安工业大学, 2023.

[4] GU W, SUN L, JIANG Z, et al. High-Performance Embedded System Design for QR Code Recognition With Deep Learning[J]. IEEE MultiMedia, 2024, 31(4): 70–78. DOI:10.1109/MMUL.2024.3459003.

[5] OTSU N. A Threshold Selection Method from Gray-Level Histograms[J]. IEEE Transactions on Systems, Man, and Cybernetics, 1979, 9(1): 62–66. DOI:10.1109/TSMC.1979.4310076.

[6] BEER D. quirc QR decoder library[CP/OL]. [2026-11-03]. https://github.com/dlbeer/quirc.

[7] AMD. Zynq 7000 SoC Technical Reference Manual UG585 Version 1.15[EB/OL]. 2026-02-06[2026-11-01]. https://docs.amd.com/r/en-US/ug585-zynq-7000-SoC-TRM.

[8] AMD. AXI Video Direct Memory Access v6.3 Product Guide PG020[EB/OL]. 2022-06-08[2026-11-03]. https://docs.amd.com/r/en-US/pg020_axi_vdma.

[9] AMD. Vitis High-Level Synthesis User Guide UG1399 Version 2025.2[EB/OL]. 2026-01-22[2026-11-03]. https://docs.amd.com/r/2025.2-English/ug1399-vitis-hls.

[10] AMD. Vivado Design Suite User Guide Implementation UG904 Version 2025.2[EB/OL]. 2025-11-20[2026-11-01]. https://docs.amd.com/r/2025.2-English/ug904-vivado-implementation.

[11] LINUX KERNEL COMMUNITY. Dynamic DMA mapping Guide Linux 6.12[EB/OL]. [2026-11-01]. https://docs.kernel.org/6.12/core-api/dma-api-howto.html.

[12] SYSTEMD PROJECT. systemd.service Service unit configuration (v257)[EB/OL]. [2026-11-03]. https://github.com/systemd/systemd/blob/v257/man/systemd.service.xml.

[13] 大恒图像. 大恒图像智能产品 [EB/OL]. 2021-06[2026-11-04]. https://sc19.gongkong.com/Upload/gongkong/technicalDataAttachment/202202/23/5f7e2d39096c4201805a9ed28361707a.pdf.

[14] 刘杰, 马力强. 一种基于 Zynq 异构平台的二维码识别系统及识别方法：CN109993018A[P]. 2019-07-09. https://patents.google.com/patent/CN109993018A/zh.

[15] AMD. Zynq-7000 SoC Product Selection Guide[EB/OL]. [2026-11-04]. https://www.amd.com/content/dam/xilinx/support/documents/selection-guides/zynq-7000-product-selection-guide.pdf.

附录 A 复现说明

本附录给出配套系统恢复及源码重建步骤。命令在最终工程包根目录执行，工具路径、设备名和网络地址按现场环境替换。

A.1 硬件与工程材料

连接 AX7021B、OV5640 与 HDMI，使用配套电源和 SD 卡。主机通过以太网建立 SSH 连接，或通过 UART 串口（115200、8N1）登录板端，摄像头对准完整场景。

工程包提供源码、构建输入及匹配发布文件；基础系统为 AX7021B PYNQ 3.1.1，板端 Python 需包含 NumPy，驱动与内核版本匹配。

quirc 适配源码位于 src/software/imported/quirc；采用版本由随报告的 assets/references/quirc_adapted_snapshot.sha256 记录六个源码文件的 SHA-256。

A.2 SD 系统备份与空白卡恢复

配套卡为约 32 GB、MBR 双分区系统，Windows 的 F 盘仅为 FAT32 启动分区；空白卡恢复需要包含 Linux 根文件系统的完整整卡镜像。

从配套卡导出整盘镜像并生成 SHA-256，校验后写入容量不小于镜像的空白卡。先核对 lsblk 中的型号和容量，将/dev/sdX 替换为实际读卡器整盘设备，并卸载该卡分区。

```
lsblk -o NAME,SIZE,MODEL,FSTYPE,MOUNTPOINTS
SD_DEVICE=/dev/sdX
sudo dd if="$SD_DEVICE" of=ax7021b_pynq_311.img bs=4M status=progress
sha256sum ax7021b_pynq_311.img > ax7021b_pynq_311.img.sha256
# 写入空白目标卡前，重新确认 SD_DEVICE 指向目标读卡器
sha256sum -c ax7021b_pynq_311.img.sha256
sudo dd if=ax7021b_pynq_311.img of="$SD_DEVICE" bs=4M conv=fsync
status=progress
sync
```

三个启动文件与已核对配套 SD 卡一致，校验值见表 A-1。

表 A-1 匹配启动文件 SHA-256

文件	SHA-256
BOOT.BIN	0f5635cd7fa7e241b378b10d00d01a1e61d0fe18f97f95c35a1a42f599be8f4f
image.ub	0cd76d257bfad2117372d0b68abc481f4ad11679ad718c53cf3a1138b56369a2
boot.scr	4c7ed60cad10872084bf727b2c9ab0eca9eab992635cf3abf67dfe7ed9cc7807

A.3 环境准备与源码校验

构建环境为 Linux x86-64、Python≥3.10、GNU make/GCC/G++/binutils 及 Vivado/Vitis 2025.2。ARM 使用 AMD GCC 13.4.0；启动打包需 XSCT、bootgen、dtc 与 mkimage。固定基础依赖从正式提交仓库的 source-deps-v1 Release 获取，模块 SDK 解压约 1.7 GB。

根据实际 Linux 构建主机设置 CC_ARM 的完整路径，并检查可执行权限与版本；其安装目录不按 Windows 工具布局推断。核验源码清单，安装测试与串口依赖，再下载并校验固定基础依赖。目标内核为 6.12.40-xilinx-ax7021b-pynq-hdmi-g5c57b943fc68。

```
export AMD_ROOT=/path/to/amd/2025.2
export VIVADO_ROOT="$AMD_ROOT/Vivado"
export VITIS_ROOT="$AMD_ROOT/Vitis"
export CC_ARM=/actual/path/to/arm-linux-gnueabihf-gcc
test -x "$CC_ARM" || exit 1
"$CC_ARM" --version
```

```
export PATH="$VIVADO_ROOT/bin:$VITIS_ROOT/bin:$PATH"
python3 -B build/scripts/verify_release.py
python3 -m venv .venv
.venv/bin/python -m pip install -r sim/requirements.txt \
  -r board/requirements-host.txt
python3 -B build/scripts/fetch_dependencies.py
python3 -B build/scripts/verify_release.py --dependencies
```

下载脚本依据 build/DEPENDENCIES.json 核对文件大小和 SHA-256，并缓存至 build/deps/。私有仓库需先使用有读取权限的账户执行 `gh auth login`；离线导入方法见 DEPENDENCIES.md。

A.4 HLS 与 Vivado 硬件重建

选择复用已导出 IP 或重新生成 HLS，两条路线择一执行；构建使用尚未占用的标签。

```
# 使用包内导出 IP 重建硬件
python3 -B build/scripts/hardware.py report-rebuild
# 重生成 HLS 时，改用以下两条命令
python3 -B build/scripts/build_hls.py --stage all
python3 -B build/scripts/hardware.py report-rebuild \
  --fitness-ip-dir build/work/hls/component/hls/impl/ip \
  --fitness-receipt build/reports/generated/hls/result.json
```

实现输出位于 build/work/，检查报告位于 build/reports/generated/。审阅本次 CDC 报告，提供绑定 routed DCP 哈希的审查 JSON 后完成资格核验，再打包启动文件。

```
python3 -B build/scripts/qualify.py report-rebuild
python3 -B build/scripts/qualify.py report-rebuild \
  --cdc-review /path/to/cdc-review.json
python3 -B build/scripts/package_boot.py report-rebuild
```

A.5 PS 应用和 Linux 驱动构建

构建 ARM 应用和驱动，核对 ELF 架构、hard-float 及模块 vermagic，再与匹配启动文件打包。运行包生成后，将匹配启动文件、运行包与安装工具装配为板端安装目录，重建产物经验证后部署。

```
python3 -B build/scripts/build_arm.py --label report-arm
python3 -B build/scripts/build_kernel.py --label report-modules
python3 -B build/scripts/r4_package_runtime.py \
  --arm-label report-arm \
  --kernel-label report-modules \
  --firmware build/outputs/boot-report-rebuild \
  --output build/outputs/runtime-report-rebuild
python3 -B build/scripts/stage_board.py \
  --firmware build/outputs/boot-report-rebuild \
  --runtime build/outputs/runtime-report-rebuild \
  --output build/outputs/board-report-rebuild
```

装配目录 build/outputs/board-report-rebuild 中包含 boot、runtime 及安装工具；正式源码中的 board/为安装工具源码，部署时使用装配后的完整目录。

A.6 安装启动与运行核验

将上节生成的安装目录复制到板端。以下账户与 IP 地址为示例，按现场替换；复制目标目录应尚不存在。安装匹配启动文件与驱动后重启，再安装运行入口。

```
# 主机执行
scp -r build/outputs/board-report-rebuild \
  xilinx@192.168.1.11:~/zynq-qr-board
ssh xilinx@192.168.1.11
```

```
# 板端执行
cd ~/zynq-qr-board
sudo python3 -B boot_files.py install
sudo reboot
# 重新 SSH 登录后，进入同一安装目录
cd ~/zynq-qr-board
uname -r
sudo sh install.sh
systemctl --version
sudo zynq-qr check
sudo zynq-qr start
sudo zynq-qr status
sudo zynq-qr logs
sudo zynq-qr stop
```

启动后 status 应为 running，停止时清除标注；再次上电仍需手动 start。安装器输出的恢复目录用于还原本次变更。

A.7 功能验证与虚拟网页仿真测试

执行以下软件、几何与 RTL 验证；Native 几何脚本使用 build/work/native 中的库。

```
.venv/bin/python -B sim/scripts/test_python.py
python3 -B build/scripts/build_arm.py --native --label native
.venv/bin/python -B sim/scripts/native_multi_geometry.py
python3 -B build/scripts/r4_rtl_test.py
python3 -m http.server 8765 --bind 0.0.0.0 --directory data
```

虚拟网页仿真测试时，浏览器访问主机 8765 端口，在副屏全屏播放包内页面；保持取景与焦距固定，参数按表 3-2 设置，保存逐组真值、识别事件和板端进度样本。

A.8 统计比对与资料保存

按第 3.4.1 节统计目标命中，按第 3.4.2 节计算加权帧率，与图 3-4 至图 3-7 比对。纯 PS 对照使用独立基线的匹配固件与运行包，保持目标内容和几何条件一致。复现时保存工具与 systemd 版本、产物校验值、配置及原始记录；技能与交互资料见第 2.4 节。

附录 B 完整硬件连接图

图 B-1 给出最终 Vivado 工程的完整 BD 连接，包含时钟、复位及辅助控制信号。本页保持 A4 竖版，仅将图形旋转 90° 排入页内；可放大核对原始实例和端口名称。

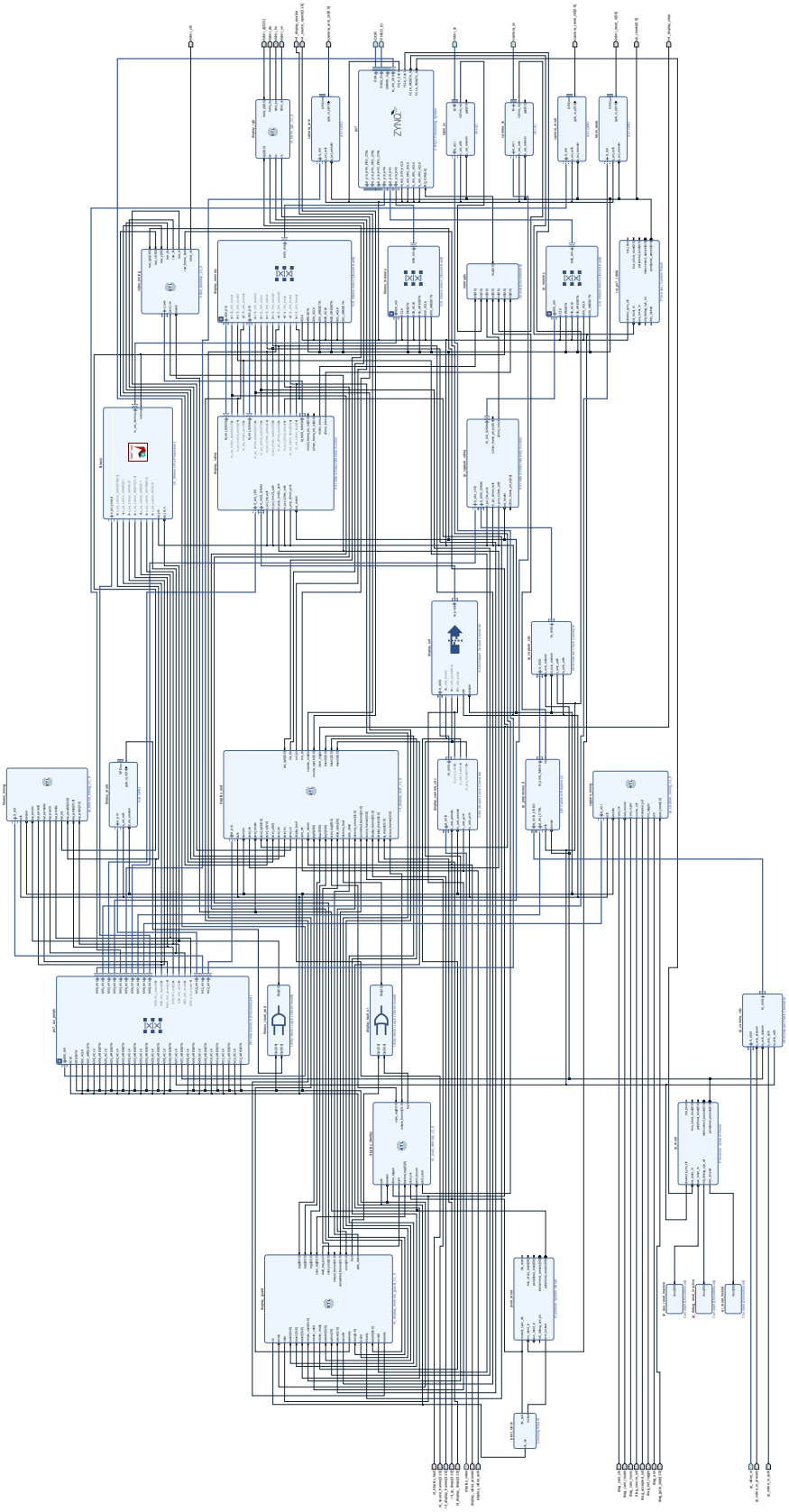


图 B-1 完整硬件连接图（图形旋转排版）